개인용 팔란티어 구축 가이드

나를 아는 AI 시스템, 이렇게 만든다

by 최성원

목차

사용법 — 이 가이드를 AI에 넣으면 알아서 만들어줍니다

이 가이드에서 만드는 것

전체 시스템 비용

필요한 도구

Part 1. 전체 시스템 구조 이해하기

왜 이게 필요한가?

1-1. 전체 그림 -- 7개 시스템이 하나로 연결된다

1-2. 각 시스템의 역할 한 줄 정리

1-3. 만들어야 하는 파일 전체 목록

Part 2. 임베딩 & 벡터 DB 구축 -- 내 메모를 AI가 이해하게 만들기

왜 이게 필요한가?

2-1. 프로젝트 폴더 생성 + Python 가상환경

2-2. 브릿지 키워드 사전 만들기

2-3. 인덱서(indexer.py) 만들기

2-4. 실행하기

Part 3. 검색 엔진 -- 중요한 메모가 위로 올라오게 만들기

왜 이게 필요한가?

3-1. 점수 공식

3-2. search.py 만들기

3-3. 검색 테스트

Part 4. MCP 서버 -- Claude가 내 메모를 도구로 쓰게 만들기

왜 이게 필요한가?

4-1. server.py 만들기

4-2. MCP 서버 등록

4-3. 사용해보기

Part 5. Nightly Synapse -- 매일 밤 AI가 자동으로 노트를 분류하고 검증한다

왜 이게 필요한가?

5-1. 프로젝트 세팅

5-2. detector.js -- 오늘 수정된 노트 감지

5-3. classifier.js -- Claude CLI로 분류 판정

분류 기준

분석할 노트

기존 핵심 노트 목록 (연결 참고용)

작업 지침

응답 형식 (JSON만 출력, 마크다운 코드블록 없이)

5-4. synapse.js -- 오케스트레이터

5-5. report.js, notify.js, run-now.js

5-6. 실행하기

5-7. 사용 흐름

Part 6. CLAUDE.md -- AI가 내 볼트를 망치지 않게 하는 규칙

왜 이게 필요한가?

6-1. CLAUDE.md 구조 (템플릿)

1. 볼트 구조 요약

2. Active Context (매 세션 시작 시 업데이트)

현재 집중 프로젝트

에너지 배분

3. 쓰기 제약 -- 볼트 오염 방지 규칙

3-1. 읽기 전용 기본값

3-2. 제안 -> 컨펌 -> 실행 (3단계 원칙)

3-3. 금지 행위

3-4. 안전장치

4. 세션 운영 규칙

4-1. 긴 대화에서 규칙 희석 방지

4-2. 컨텍스트 윈도우 관리

5. 판단 기준

6-2. Ground Truth (선택사항)

6-3. 메모리 시스템 (선택사항)

Part 7. 텔레그램 파이프라인 -- 휴대폰으로 말하면 자동 저장

왜 이게 필요한가?

7-1. 전체 흐름

7-2. Step 1: 텔레그램 봇 생성 (5분)

7-3. Step 2: GitHub 저장소 + 토큰 (10분)

7-4. Step 3: AWS 설정 (20분)

분류 기준

브릿지 키워드 매핑

출력 형식 (JSON)

7-5. Step 4: Webhook 연결 (1분)

7-6. Step 5: 배포 스크립트

7-7. 비용 구조

Part 8. 동기화 & 백업 -- 데이터를 안전하게 지키기

왜 이게 필요한가?

8-1. GitHub 양방향 동기화 스크립트

8-2. 일일 백업 스크립트

8-3. 자동 실행 (macOS LaunchAgent)

Part 9. 개발 하네스 -- AI가 코드를 짤 때 품질을 잡아주는 시스템 (선택)

왜 이게 필요한가?

9-1. 하네스 구성 요소

9-2. 훅 설정 (settings.json)

9-3. 규칙 파일

Secret Management

Core Principles

Immutability

9-4. 사용 방법

Part 10. 일상 운영 가이드 -- 매일 이것만 하면 된다

왜 이게 필요한가?

10-1. 매일 아침 (5분)

10-2. 메모를 쓰거나 수정했을 때

10-3. 상황별 대응 체크리스트

10-4. 주간 체크 (선택)

10-5. 전체 파일 위치 한눈에 보기

마치며

개인용 팔란티어 구축 가이드

— 나를 아는 AI 시스템, 이렇게 만든다.

> 이 문서는 무료PDF "AI 시대 나침반"의 CHAPTER 3에서 언급한 개인용 팔란티어의 구체적인 구축 방법을 다루는 별도 가이드다. 이 가이드대로 따라하면 누구나 자기만의 Palantir를 만들 수 있다.

사용법 — 이 가이드를 AI에 넣으면 알아서 만들어줍니다

이 문서는 두 가지 방식으로 사용할 수 있습니다:

방법 1: Claude Code에 통째로 시키기 (추천)

Claude Code 터미널에서 이렇게 말하면 됩니다:

```
 이 가이드를 읽고, Part 2부터 순서대로 구축해줘.
내 옵시디언 볼트 경로는 ~/Documents/Obsidian Vault/ 이야.
각 Part를 완료할 때마다 확인하기 체크리스트를 돌려서 결과를 보여줘.
```

AI가 이 문서의 코드를 읽고, 당신의 볼트 경로에 맞게 수정해서, 파일을 생성하고, 설정까지 해줍니다.

방법 2: 직접 따라하기

각 Part의 코드를 복사해서 터미널에 붙여넣으면 됩니다. `{여기에_본인_값}` 형태의 플레이스홀더만 본인 정보로 바꾸면 됩니다.

보안 주의: 이 가이드에 포함된 코드에는 API 키, 토큰, 비밀번호가 없습니다. `{YOUR_TOKEN}`, `{YOUR_CHAT_ID}` 같은 부분은 반드시 본인 값으로 교체하세요.

이 가이드에서 만드는 것

7가지 시스템을 하나씩 구축한다:

1. 내 맥락에 맞는 답변을 해주는 AI (MCP 서버 + 벡터 검색)
2. 새 메모를 쓰면 자동으로 업데이트되는 임베딩 (증분 인덱싱)
3. 휴대폰으로 말하면 자동 분류 + 저장되는 파이프라인 (텔레그램 → AWS → GitHub)
4. 매일 밤 AI가 내 노트를 자동 분류하고 교차검증하는 시스템 (Nightly Synapse)
5. AI가 내 볼트를 망치지 않게 하는 규칙 (CLAUDE.md + Ground Truth)
6. 동기화 + 백업 자동화 (GitHub 양방향 + 일일 백업)
7. AI가 코드를 짤 때 품질을 자동으로 잡아주는 하네스 (선택)

전체 시스템 비용

항목	비용
옵시디언	무료
Claude Code (CLI)	Anthropic 구독
텔레그램 파이프라인 (AWS Lambda + Bedrock)	월 ~$2 (100건 기준)
Nightly Synapse (Claude CLI 토큰)	Claude 구독에 포함
GitHub 동기화	무료
백업	무료
합계	**월 ~$2 + Claude 구독**

필요한 도구

도구	용도	비용
옵시디언	노트 앱	무료
Claude Code (CLI)	AI 터미널 도구	Anthropic 구독
Python 3.x	임베딩/검색/MCP 서버	무료
Node.js	Nightly Synapse	무료
AWS 계정	Lambda + Bedrock + DynamoDB	월 ~$2
GitHub 계정	볼트 동기화 저장소	무료
텔레그램	모바일 메모 입력	무료
터미널 기본 사용법	복붙만 하면 됨	--

Part 1. 전체 시스템 구조 이해하기

왜 이게 필요한가?

전체 그림을 모르면 각 부품이 왜 필요한지 이해가 안 된다. 먼저 "내가 뭘 만드는지" 그림을 그려라. 그 다음에 하나씩 만든다.

1-1. 전체 그림 -- 7개 시스템이 하나로 연결된다

```
 [입력]
  텔레그램 메모 -> Lambda(Bedrock Claude) -> GitHub -> 볼트 (To_Process_Daily/)
  옵시디언 직접 작성 -> 볼트

[처리 -- 매일 자동]
  21:30 Nightly Synapse
    +-- detector.js -- 오늘 수정된 노트 감지
    +-- classifier.js -- Claude CLI로 PARA/Slip-Box 분류 판정
    +-- report.js -- 분류 리포트 생성 -> Inbox/
    +-- notify.js -- macOS 알림

  22:30 Daily Audit
    +-- auditor.js -- Claude 작업 교차검증
    +-- git diff + 세션 로그 분석 -> Inbox/

[검색 -- 실시간]
  indexer.py -> 벡터화 -> LanceDB (증분 인덱싱)
  search.py -> 4중 가중치 검색
  server.py -> MCP 서버 -> Claude Code가 도구로 사용
    "내 핵심 가치 알려줘" -> 관련 노트 찾아서 답변

[보호]
  CLAUDE.md -- 볼트 운영 규칙 (읽기 전용 기본값, 3단계 컨펌)
  Ground Truth -- AI 자동 분류 조건표 (If-Then 로직)

[동기화]
  sync_from_github.sh -> 양방향 (push + pull, 30분마다)
  backup_vault.sh -> 매일 21:30 백업 (7일 보관)

[개발 품질]
  Claude Code 하네스 (에이전트 + 스킬 + 규칙 + 훅)
```

1-2. 각 시스템의 역할 한 줄 정리

#	시스템	한 줄 설명	위치
1	MCP 서버	내 메모를 의미로 검색하고, Claude가 도구로 쓸 수 있게 연결	`~/obsidian-mcp-server/`
2	텔레그램 파이프라인	걸어가면서 메모하면 자동으로 볼트에 저장	`~/telegram-obsidian-pipeline/`
3	Nightly Synapse	매일 밤 자동으로 노트 분류 제안 + Claude 작업 교차검증	`볼트/scripts/nightly-synapse/`
4	CLAUDE.md + Ground Truth	AI가 볼트를 망치지 않게 하는 절대 규칙	볼트 루트 + AoR 폴더
5	동기화 + 백업	GitHub 양방향 30분마다 + 일일 백업 7일 보관	`~/ObsidianBackup/`
6	Claude Code 하네스	AI가 코드를 짤 때 보안/품질 자동 점검	`~/.claude/`
7	LaunchAgents	위 시스템들의 자동 스케줄 관리 (macOS)	`~/Library/LaunchAgents/`

1-3. 만들어야 하는 파일 전체 목록

```
~/obsidian-mcp-server/
  +-- indexer.py          # 증분 인덱싱 엔진
  +-- search.py           # 4중 가중치 검색 엔진
  +-- bridge_keywords.py  # 브릿지 키워드 사전
  +-- server.py           # MCP 서버
  +-- vault.lancedb/      # 벡터 DB (자동 생성)
  +-- .index_state.json   # 증분 비교용 상태 (자동 생성)
  +-- venv/               # Python 가상환경

~/telegram-obsidian-pipeline/
  +-- lambda_function.py  # Lambda 핵심 코드
  +-- deploy.sh           # 배포 스크립트

볼트/scripts/nightly-synapse/
  +-- synapse.js          # 오케스트레이터
  +-- detector.js         # 오늘 수정된 노트 감지
  +-- classifier.js       # Claude CLI 분류 판정
```

```
  +-- report.js           # 리포트 생성
  +-- auditor.js          # 교차검증
  +-- notify.js           # macOS 알림
  +-- scheduler.js        # cron 스케줄러
  +-- run-now.js          # 수동 Synapse 실행
  +-- run-audit.js        # 수동 Audit 실행
  +-- package.json        # Node.js 의존성

~/ObsidianBackup/
  +-- sync_from_github.sh   # GitHub 양방향 동기화
  +-- backup_vault.sh       # 일일 백업

~/.claude/
  +-- .mcp.json             # MCP 서버 등록
  +-- settings.json         # 훅 설정

볼트 루트/
  +-- CLAUDE.md             # 볼트 운영 규칙
```

확인하기

- [ ] 옵시디언이 설치되어 있다
- [ ] 터미널(맥: Terminal 또는 iTerm, 윈도우: PowerShell)을 열 수 있다
- [ ] Python 3이 설치되어 있다 (`python3 --version`)
- [ ] Node.js가 설치되어 있다 (`node --version`)
- [ ] GitHub 계정이 있다
- [ ] AWS 계정이 있다 (텔레그램 파이프라인용)
- [ ] Claude Code가 설치되어 있다 (Anthropic 구독)

Part 2. 임베딩 & 벡터 DB 구축 -- 내 메모를 AI가 이해하게 만들기

왜 이게 필요한가?

옵시디언의 기본 검색은 키워드 매칭이다. "성장"이라고 검색하면 "성장"이라는 글자가 들어간 노트만 나온다. "발전", "향상", "진화"는 안 나온다. 임베딩은 텍스트를 숫자(벡터)로 바꿔서, 의미가 비슷하면 찾아준다. 이게 개인용 팔란티어의 기반이다.

2-1. 프로젝트 폴더 생성 + Python 가상환경

```
 # 1. 프로젝트 폴더 생성
mkdir -p ~/obsidian-mcp-server
cd ~/obsidian-mcp-server

# 2. Python 가상환경 생성 (시스템 파이썬과 격리)
python3 -m venv venv

# 3. 가상환경 활성화
source venv/bin/activate

# 4. 필요한 패키지 설치
pip install sentence-transformers lancedb mcp[cli]

# 5. 가상환경 비활성화 (설치 끝났으니 나옴)
deactivate
```

> **왜 가상환경을 쓰는가?** 시스템 파이썬에 직접 설치하면 다른 프로젝트와 라이브러리가 충돌할 수 있다. 가상환경(venv)은 이 프로젝트 전용 독립된 파이썬 환경이다. 지우고 싶으면 폴더만 삭제하면 끝.

2-2. 브릿지 키워드 사전 만들기

이건 나중에 검색 엔진과 텔레그램 파이프라인에서도 쓰이는 핵심 사전이다. "브릿지 키워드"란 당신의 노트를 관통하는 핵심 주제들이다. 각 주제마다 "이런 단어가 나오면 이 주제에 해당한다"는 신호 단어를 정의한다.

`~/obsidian-mcp-server/bridge_keywords.py` 파일을 만들어라:

```
 """
브릿지 키워드 사전
- 키: 당신의 핵심 주제 이름 (옵시디언 [[]] 링크명과 동일)
- 값: 해당 주제를 감지하는 신호 단어 리스트

수정 후 반드시 indexer.py를 다시 실행해야 반영됩니다.
"""

BRIDGE_KEYWORDS = {
    # === 당신의 핵심 주제를 여기에 정의하세요 ===
    #
    # 형식: "주제_이름": ["신호단어1", "신호단어2", "신호단어3"]
    #
    # 예시:
    "시스템_사고": ["시스템", "구조", "설계", "자동화"],
    "의사결정": ["판단", "선택", "의사결정", "결정 기준"],
    "습관_설계": ["습관", "루틴", "반복", "매일"],
    "독서_확장": ["독서", "읽고 나서", "인식이 바뀌"],
    "스토리텔링": ["스토리", "서사", "이야기 구조"],
    "브랜딩": ["포지셔닝", "브랜딩", "차별화"],
    "피드백_루프": ["피드백", "반복", "루프", "개선 사이클"],
    "에너지_관리": ["감정", "에너지", "번아웃"],
    #
    # 10~30개 사이가 적당하다.
    # 너무 적으면 검색 보너스가 약하고, 너무 많으면 노이즈가 생긴다.
}
```

> **왜 이렇게 하는가?** 벡터 검색은 "의미"를 잡지만, 당신만의 분류 체계까지는 모른다. 브릿지 키워드를 통해 "이 노트와 저 노트가 같은 주제다"라는 연결을 AI에게 가르치는 거다. 당신의 노트를 관통하는 핵심 키워드 10~30개를 정의하라.

2-3. 인덱서(indexer.py) 만들기

`~/obsidian-mcp-server/indexer.py` 파일을 만들어라:

```
 """
옵시디언 노트 인덱서 (증분 인덱싱)
- 변경/추가된 노트만 업데이트, 삭제된 노트는 DB에서 제거
- --full 플래그로 전체 재인덱싱 가능
```

```
"""

import os
import re
import sys
import time
import json
import lancedb
from pathlib import Path
from sentence_transformers import SentenceTransformer
from bridge_keywords import BRIDGE_KEYWORDS

# ==============================================
# 설정 (여기만 수정하세요)
# ==============================================
VAULT_PATH = "{여기에_본인_볼트_절대경로}"   # 예: "/Users/me/Documents/Obsidian Vault"
DB_PATH = os.path.expanduser("~/obsidian-mcp-server/vault.lancedb")
TABLE_NAME = "notes"
EMBEDDING_MODEL = "paraphrase-multilingual-MiniLM-L12-v2"
STATE_FILE = os.path.expanduser("~/obsidian-mcp-server/.index_state.json")

# 너무 긴 노트는 잘라서 저장 (토큰 제한 방지)
MAX_CHARS = 3000


# ==============================================
# 1. 폴더별 가중치 -- 중요한 노트가 검색에서 위로 올라온다
# ==============================================
def get_folder_weight(rel_path: str, filename: str) -> float:
    """
    당신의 폴더 구조에 맞게 수정하세요.
    핵심 메모 폴더 -> 3.0, 프로젝트 폴더 -> 2.0, 나머지 -> 1.0
    """
    if rel_path.startswith("0. Slip-Box/") or "P1E" in filename:
        return 3.0
    elif rel_path.startswith("1. Project/"):
        return 2.0
    else:
        return 1.0


# ==============================================
# 2. 브릿지 키워드 감지
# ==============================================
def detect_bridge_keywords(text: str) -> str:
    text_lower = text.lower()
    matched = []
    for kw_name, signals in BRIDGE_KEYWORDS.items():
        for signal in signals:
            if signal.lower() in text_lower:
                matched.append(kw_name)
                break
    return ",".join(matched)
```

```
# ==============================================
# 3. 마크다운 정리 -- 노이즈 제거
# ==============================================
def clean_markdown(text: str) -> str:
    text = re.sub(r"^---\n.*?\n---\n?", "", text, flags=re.DOTALL)  # 프론트매터 제거
    text = re.sub(r"!\[\[.*?\]\]", "", text)                         # 이미지 임베드 제거
    text = re.sub(r"\[\[([^|\]]*\|)?([^\]]+)\]\]", r"\2", text)     # [[링크]] -> 텍스트
    text = re.sub(r"!\[.*?\]\(.*?\)", "", text)                      # 마크다운 이미지 제거
    text = re.sub(r"<[^>]+>", "", text)                              # HTML 태그 제거
    text = re.sub(r"\n{3,}", "\n\n", text)                           # 과도한 빈 줄 정리
    return text.strip()


# ==============================================
# 4. 볼트 스캔 -- 모든 .md 파일과 수정시간 수집
# ==============================================
def scan_vault(vault_path: str) -> dict[str, float]:
    vault = Path(vault_path)
    # 무시할 폴더 (본인 볼트에 맞게 수정)
    skip_dirs = {".obsidian", ".trash", ".smart-env", "Attachments", ".git"}
    files = {}

    for md_file in vault.rglob("*.md"):
        if any(part in skip_dirs for part in md_file.parts):
            continue
        rel_path = str(md_file.relative_to(vault))
        files[rel_path] = md_file.stat().st_mtime

    return files


# ==============================================
# 5. 상태 관리 (이전 인덱싱 시점의 mtime 기록)
# ==============================================
def load_state() -> dict[str, float]:
    if os.path.exists(STATE_FILE):
        with open(STATE_FILE, "r") as f:
            return json.load(f)
    return {}


def save_state(state: dict[str, float]):
    with open(STATE_FILE, "w") as f:
        json.dump(state, f)


# ==============================================
# 6. 변경 사항 계산 -- 추가/수정/삭제 분류
# ==============================================
def compute_changes(current: dict[str, float], previous: dict[str, float]):
    added = []
    modified = []
    deleted = []

    for path, mtime in current.items():
```

```
        if path not in previous:
            added.append(path)
        elif mtime > previous[path]:
            modified.append(path)

    for path in previous:
        if path not in current:
            deleted.append(path)

    return added, modified, deleted


# ==============================================
# 7. 노트 하나 처리 -> 데이터 dict
# ==============================================
def process_note(vault_path: str, rel_path: str) -> dict | None:
    full_path = Path(vault_path) / rel_path
    try:
        text = full_path.read_text(encoding="utf-8").strip()
    except Exception:
        return None

    if not text:
        return None

    clean = clean_markdown(text)
    if len(clean) < 10:
        return None

    filename = full_path.stem
    weight = get_folder_weight(rel_path, filename)
    bridge_kws = detect_bridge_keywords(clean)

    return {
        "filename": filename,
        "path": rel_path,
        "text": clean[:MAX_CHARS],
        "weight": weight,
        "bridge_keywords": bridge_kws,
    }


# ==============================================
# 8. 전체 재인덱싱
# ==============================================
def full_reindex(vault_path: str, current_files: dict[str, float]):
    print("[ 전체 재인덱싱 모드 ]")
    notes = []
    for rel_path in current_files:
        note = process_note(vault_path, rel_path)
        if note:
            notes.append(note)

    print(f"  -> {len(notes)}개 노트 발견")
    if not notes:
```

```
        print("노트를 찾지 못했습니다.")
        return

    print(f"임베딩 모델 로딩 중: {EMBEDDING_MODEL}")
    model = SentenceTransformer(EMBEDDING_MODEL)

    print(f"{len(notes)}개 노트 임베딩 중...")
    texts = [n["text"] for n in notes]
    vectors = model.encode(texts, show_progress_bar=True, batch_size=64)

    db = lancedb.connect(DB_PATH)

    data = []
    for i, note in enumerate(notes):
        note["vector"] = vectors[i].tolist()
        data.append(note)

    # 기존 테이블이 있으면 삭제 후 재생성
    existing = db.list_tables()
    table_names = existing.tables if hasattr(existing, "tables") else list(existing)
    if TABLE_NAME in table_names:
        db.drop_table(TABLE_NAME)

    db.create_table(TABLE_NAME, data=data)
    print(f"완료! {len(notes)}개 노트가 LanceDB에 저장되었습니다.")
    save_state(current_files)


# ==============================================
# 9. 증분 인덱싱 -- 변경분만 처리
# ==============================================
def incremental_index(vault_path: str, current_files: dict[str, float],
                      added: list, modified: list, deleted: list):
    print("[ 증분 인덱싱 모드 ]")
    print(f"  -> 추가: {len(added)}개 / 수정: {len(modified)}개 / 삭제: {len(deleted)}개")

    to_process = added + modified

    if not to_process and not deleted:
        print("  -> 변경 사항 없음. 스킵.")
        return

    db = lancedb.connect(DB_PATH)
    table = db.open_table(TABLE_NAME)

    # 삭제 처리
    if deleted:
        for path in deleted:
            table.delete(f'path = "{path.replace(chr(34), "")}"')
        print(f"  -> {len(deleted)}개 노트 DB에서 제거 완료")

    # 추가/수정 처리
    if to_process:
        notes = []
        for rel_path in to_process:
```

```
            note = process_note(vault_path, rel_path)
            if note:
                notes.append(note)

        if notes:
            print(f"임베딩 모델 로딩 중: {EMBEDDING_MODEL}")
            model = SentenceTransformer(EMBEDDING_MODEL)

            print(f"{len(notes)}개 노트 임베딩 중...")
            texts = [n["text"] for n in notes]
            vectors = model.encode(texts, show_progress_bar=True, batch_size=64)

            # 수정된 노트는 기존 레코드 삭제 후 재삽입
            for note in notes:
                safe_path = note["path"].replace(chr(34), "")
                table.delete(f'path = "{safe_path}"')

            data = []
            for i, note in enumerate(notes):
                note["vector"] = vectors[i].tolist()
                data.append(note)

            table.add(data)
            print(f"  -> {len(data)}개 노트 임베딩 완료")

    save_state(current_files)


# ============================================
# 실행
# ============================================
if __name__ == "__main__":
    start = time.time()
    force_full = "--full" in sys.argv

    print(f"볼트 경로: {VAULT_PATH}")
    print("볼트 스캔 중...")
    current_files = scan_vault(VAULT_PATH)
    print(f"  -> {len(current_files)}개 .md 파일 발견")

    previous_state = load_state()

    db = lancedb.connect(DB_PATH)
    existing = db.list_tables()
    table_names = existing.tables if hasattr(existing, "tables") else list(existing)
    need_full = force_full or not previous_state or TABLE_NAME not in table_names

    if need_full:
        if force_full:
            print("  -> --full 플래그 감지. 전체 재인덱싱 실행.")
        elif not previous_state:
            print("  -> 첫 실행. 전체 재인덱싱 실행.")
        else:
            print("  -> DB 테이블 없음. 전체 재인덱싱 실행.")
        full_reindex(VAULT_PATH, current_files)
```

```
    else:
        added, modified, deleted = compute_changes(current_files, previous_state)
        incremental_index(VAULT_PATH, current_files, added, modified, deleted)

    elapsed = time.time() - start
    print(f"\n총 소요 시간: {elapsed:.1f}초")
```

2-4. 실행하기

```
 # 첫 실행 -- 전체 인덱싱 (볼트 크기에 따라 10~60초)
cd ~/obsidian-mcp-server
./venv/bin/python indexer.py

# 이후 -- 증분 인덱싱 (변경분만 처리, 보통 0.1~2초)
./venv/bin/python indexer.py

# 임베딩 모델을 바꿀 때만 -- 전체 재인덱싱
./venv/bin/python indexer.py --full
```

확인하기

실행 후 이런 출력이 나오면 성공이다:

```
 볼트 경로: /Users/me/Documents/Obsidian Vault
볼트 스캔 중...
  -> 500개 .md 파일 발견
[ 전체 재인덱싱 모드 ]
  -> 498개 노트 발견
임베딩 모델 로딩 중: paraphrase-multilingual-MiniLM-L12-v2
498개 노트 임베딩 중...
완료! 498개 노트가 LanceDB에 저장되었습니다.

총 소요 시간: 15.3초
```

`vault.lancedb/` 폴더가 생겼는지 확인:

```
 ls -la ~/obsidian-mcp-server/vault.lancedb/
```

문제가 생기면

증상	원인	해결
`ModuleNotFoundError: No module named 'sentence_transformers'`	가상환경 밖에서 실행	`./venv/bin/python indexer.py` 로 실행 (venv 경로 포함)
`VAULT_PATH` 관련 에러	경로가 틀림	볼트의 절대 경로를 정확히 입력 (공백 주의)
노트가 0개로 나옴	skip_dirs에 볼트 폴더가 걸림	`skip_dirs` 세트를 본인 환경에 맞게 수정
임베딩 모델 다운로드 느림	첫 실행 시 모델 다운로드	정상. 한 번만 다운로드됨 (~100MB)

Part 3. 검색 엔진 -- 중요한 메모가 위로 올라오게 만들기

왜 이게 필요한가?

단순 벡터 검색은 "의미가 비슷한 것"만 찾는다. 문제는 핵심 메모(Slip-Box)와 잡메모(일간 노트)가 같은 가중치를 받는다는 거다. "내 핵심 가치"를 검색했는데 휘발성 메모가 1등으로 나오면 쓸모없다. 4중 가중치로 "내 기준의 중요도"를 반영한다.

3-1. 점수 공식

```
최종점수 = (벡터유사도 + 제목보너스 + 브릿지보너스) x 폴더가중치
```

점수 요소	설명	범위
벡터유사도	의미적 거리 계산 `1/(1+L2거리)`	0~1.0
제목보너스	파일명에 검색어가 포함되면	+0.3
브릿지보너스	쿼리와 노트가 같은 키워드 공유 시, 키워드당	+0.15 (최대 +0.45)
폴더가중치	핵심 메모는 3배, 프로젝트는 2배	x1.0~3.0

점수 계산 예시:

```
 쿼리: "의사결정의 본질"

노트 A (Slip-Box, 핵심 노트):
  벡터유사도: 0.72
  제목보너스: +0.3 (파일명에 "의사결정" 포함)
  브릿지보너스: +0.15 (의사결정 키워드 공유)
  소계: 1.17
  폴더가중치: x3.0 (Slip-Box)
  -> 최종점수: 3.51
```

```
노트 B (일간 노트, 가벼운 메모):
  벡터유사도: 0.80 (의미는 더 비슷함!)
  제목보너스: 0
  브릿지보너스: 0
  소계: 0.80
  폴더가중치: x1.0
  -> 최종점수: 0.80

결과: 노트 A가 1등. 의미만으로는 B가 높지만, 가중치 덕분에 핵심 노트가 올라온다.
```

3-2. search.py 만들기

`~/obsidian-mcp-server/search.py` 파일을 만들어라:

```
 """
옵시디언 노트 검색기
- LanceDB 벡터 검색 (의미 기반)
- 파일명/제목 키워드 매칭 보너스
- 브릿지 키워드 겹침 보너스
- 폴더 가중치 (핵심 메모 -> 3배, 프로젝트 -> 2배)
"""

import os
import lancedb
from sentence_transformers import SentenceTransformer
from bridge_keywords import BRIDGE_KEYWORDS

# ==============================================
# 설정
# ==============================================
DB_PATH = os.path.expanduser("~/obsidian-mcp-server/vault.lancedb")
TABLE_NAME = "notes"
EMBEDDING_MODEL = "paraphrase-multilingual-MiniLM-L12-v2"

# 점수 가중치 (필요에 따라 조절)
TITLE_MATCH_BONUS = 0.3           # 파일명에 검색어 포함 시 보너스
BRIDGE_KEYWORD_BONUS = 0.15       # 브릿지 키워드 1개 겹칠 때마다
BRIDGE_KEYWORD_MAX_BONUS = 0.45   # 브릿지 키워드 보너스 최대 (3개분)


# ==============================================
# 모델 & DB 싱글턴 (한 번만 로딩)
# ==============================================
_model = None
_table = None


def _get_model():
    global _model
    if _model is None:
        _model = SentenceTransformer(EMBEDDING_MODEL)
```

```
    return _model


def _get_table():
    global _table
    if _table is None:
        db = lancedb.connect(DB_PATH)
        _table = db.open_table(TABLE_NAME)
    return _table


# ==============================================
# 브릿지 키워드 감지 (쿼리용)
# ==============================================
def detect_query_bridge_keywords(query: str) -> set:
    query_lower = query.lower()
    matched = set()
    for kw_name, signals in BRIDGE_KEYWORDS.items():
        for signal in signals:
            if signal.lower() in query_lower:
                matched.add(kw_name)
                break
    return matched


# ==============================================
# 검색 함수 -- 이게 핵심이다
# ==============================================
def search(query: str, top_k: int = 5) -> list[dict]:
    """
    최종 점수 = (벡터유사도 + 제목보너스 + 브릿지보너스) x 폴더가중치
    """
    model = _get_model()
    table = _get_table()

    # 질문 -> 벡터 변환
    query_vector = model.encode(query).tolist()

    # LanceDB에서 넉넉히 가져오기 (리랭킹 후 top_k로 자름)
    fetch_count = max(top_k * 10, 100)
    raw_results = (
        table.search(query_vector)
        .limit(fetch_count)
        .to_list()
    )

    # 쿼리에서 브릿지 키워드 감지
    query_bridge_kws = detect_query_bridge_keywords(query)
    query_words = query.lower().split()

    # 점수 계산 & 리랭킹
    scored = []

    for row in raw_results:
        # (1) 벡터 유사도: L2 거리 -> 유사도 변환
```

```
        distance = row.get("_distance", 1.0)
        similarity = 1.0 / (1.0 + distance)

        # (2) 제목 보너스: 검색어 단어가 파일명에 포함되면 가산
        filename_lower = row["filename"].lower()
        title_bonus = 0.0
        for word in query_words:
            if len(word) >= 2 and word in filename_lower:
                title_bonus = TITLE_MATCH_BONUS
                break

        # (3) 브릿지 키워드 보너스: 쿼리와 노트가 공유하는 키워드 수
        note_bridge_kws = set(row.get("bridge_keywords", "").split(",")) - {""}
        overlap_count = len(query_bridge_kws & note_bridge_kws)
        bridge_bonus = min(overlap_count * BRIDGE_KEYWORD_BONUS, BRIDGE_KEYWORD_MAX_B

        # (4) 폴더 가중치 (인덱싱 시 저장된 값)
        folder_weight = row.get("weight", 1.0)

        # 최종 점수
        final_score = (similarity + title_bonus + bridge_bonus) * folder_weight

        scored.append({
            "filename": row["filename"],
            "path": row["path"],
            "text": row["text"],
            "score": round(final_score, 4),
            "_detail": {
                "similarity": round(similarity, 4),
                "title_bonus": title_bonus,
                "bridge_bonus": bridge_bonus,
                "bridge_overlap": overlap_count,
                "folder_weight": folder_weight,
            },
        })

    # 점수 높은 순으로 정렬 -> 상위 top_k개 반환
    scored.sort(key=lambda x: x["score"], reverse=True)
    return scored[:top_k]


# ============================================
# 테스트 실행
# ============================================
if __name__ == "__main__":
    test_queries = [
        "의사결정의 본질",
        "좋은 습관을 만드는 방법",
        "AI와 함께 일하기",
    ]

    print("=" * 60)
    print("옵시디언 노트 검색 테스트")
    print(f"점수 = (유사도 + 제목보너스 + 브릿지보너스) x 폴더가중치")
    print("=" * 60)
```

```
    for q in test_queries:
        print(f"\n질문: \"{q}\"")
        q_kws = detect_query_bridge_keywords(q)
        if q_kws:
            print(f"   감지된 브릿지 키워드: {', '.join(q_kws)}")
        print("-" * 50)

        results = search(q, top_k=5)
        for i, r in enumerate(results, 1):
            d = r["_detail"]
            print(f"  {i}. [{r['score']:.4f}] {r['filename']}")
            print(f"     유사도={d['similarity']:.4f} "
                  f"제목+{d['title_bonus']:.1f} "
                  f"브릿지+{d['bridge_bonus']:.2f} "
                  f"x{d['folder_weight']:.0f}배")
            print(f"     경로: {r['path']}")
        print()
```

3-3. 검색 테스트

```
 cd ~/obsidian-mcp-server
./venv/bin/python search.py
```

확인하기

이런 출력이 나오면 성공:

```
 ============================================================
옵시디언 노트 검색 테스트
점수 = (유사도 + 제목보너스 + 브릿지보너스) x 폴더가중치
============================================================

질문: "의사결정의 본질"
--------------------------------------------------
  1. [2.8340] 핵심 메모 이름
     유사도=0.7213 제목+0.3 브릿지+0.15 x3배
     경로: 0. Slip-Box/...
```

폴더 가중치가 높은 노트가 상위에 올라왔는지 확인하라.

문제가 생기면

증상	원인	해결
`FileNotFoundError: vault.lancedb`	indexer.py를 아직 안 돌림	Part 2 먼저 완료
검색 결과가 엉뚱함	브릿지 키워드가 맞지 않음	`bridge_keywords.py` 의 신호 단어를 본인 노트에 맞게 수정
핵심 노트가 안 올라옴	`get_folder_weight` 함수가 본인 폴더 구조와 안 맞음	indexer.py의 폴더 조건을 수정 후 `--full` 재인덱싱

Part 4. MCP 서버 -- Claude가 내 메모를 도구로 쓰게 만들기

왜 이게 필요한가?

검색 엔진을 만들었지만, 아직 터미널에서만 쓸 수 있다. MCP(Model Context Protocol)는 AI에게 새로운 능력을 붙여주는 규격이다. 이걸로 Claude Code에 "내 노트 검색"이라는 도구를 추가한다. Claude에게 "내 핵심 가치 알려줘"라고 하면, Claude가 자동으로 관련 노트를 찾아서 그걸 기반으로 답변한다.

4-1. server.py 만들기

`~/obsidian-mcp-server/server.py` 파일을 만들어라:

```
 """
옵시디언 노트 검색 MCP 서버
- Claude Code에서 도구(tool)로 사용 가능
- 질문을 받으면 관련 노트를 찾아서 반환
"""

import os

from mcp.server.fastmcp import FastMCP
from search import search, detect_query_bridge_keywords

VAULT_PATH = "{여기에_본인_볼트_절대경로}"

# MCP 서버 생성
mcp = FastMCP("obsidian-search")


@mcp.tool()
def search_notes(query: str, top_k: int = 5) -> str:
    """
    옵시디언 볼트에서 질문과 관련된 노트를 검색합니다.

    점수 계산: (벡터유사도 + 제목보너스 + 브릿지키워드보너스) x 폴더가중치
```

```
    Args:
        query: 검색할 질문 (예: "의사결정의 본질", "습관을 만드는 방법")
        top_k: 반환할 노트 수 (기본 5개, 최대 20개)
    """
    query = query.strip()
    if not query:
        return "검색어를 입력해주세요."

    top_k = min(max(top_k, 1), 20)
    results = search(query, top_k=top_k)

    if not results:
        return "관련 노트를 찾지 못했습니다."

    query_kws = detect_query_bridge_keywords(query)

    lines = []
    lines.append(f"\"{query}\" 검색 결과 ({len(results)}개)")
    if query_kws:
        lines.append(f"감지된 브릿지 키워드: {', '.join(query_kws)}")
    lines.append("")

    for i, r in enumerate(results, 1):
        d = r["_detail"]
        tags = []
        if d["folder_weight"] == 3.0:
            tags.append("핵심노트")
        elif d["folder_weight"] == 2.0:
            tags.append("프로젝트")
        if d["title_bonus"] > 0:
            tags.append("제목매칭")
        if d["bridge_overlap"] > 0:
            tags.append(f"브릿지x{d['bridge_overlap']}")
        tag_str = " ".join(tags)

        lines.append(f"## {i}. [{r['score']:.4f}] {r['filename']}")
        lines.append(f"경로: {r['path']}")
        if tag_str:
            lines.append(tag_str)
        lines.append("")
        # 본문 미리보기 (앞 500자)
        preview = r["text"][:500].replace("\n", " ").strip()
        lines.append(f"> {preview}")
        lines.append("")

    return "\n".join(lines)


@mcp.tool()
def get_note(path: str) -> str:
    """
    옵시디언 노트의 전체 내용을 읽어옵니다.
    search_notes로 찾은 노트의 경로(path)를 넣으면 전문을 반환합니다.

    Args:
```

```
        path: 노트 경로 (예: "0. Slip-Box/철학/메모.md")
    """
    path = path.strip()
    if not path:
        return "파일 경로를 입력해주세요."

    full_path = os.path.realpath(os.path.join(VAULT_PATH, path))

    # 보안: 볼트 바깥 접근 차단
    if not full_path.startswith(os.path.realpath(VAULT_PATH)):
        return "볼트 외부 파일에는 접근할 수 없습니다."

    if not os.path.exists(full_path):
        return f"파일을 찾을 수 없습니다: {path}"

    with open(full_path, "r", encoding="utf-8") as f:
        content = f.read()

    return f"{path}\n\n{content}"


if __name__ == "__main__":
    mcp.run()
```

> **왜 보안 코드가 있는가?** `os.path.realpath`로 심볼릭 링크 우회를 차단한다. 이 코드가 없으면 AI가 `../../etc/passwd` 같은 경로로 볼트 밖 파일을 읽을 수 있다. 개인 시스템이라도 보안 습관은 중요하다.

4-2. MCP 서버 등록

`~/.claude/.mcp.json` 파일을 만들거나 수정하라:

```
{
  "mcpServers": {
    "obsidian-search": {
      "command": "{여기에_본인_홈경로}/obsidian-mcp-server/venv/bin/python",
      "args": ["{여기에_본인_홈경로}/obsidian-mcp-server/server.py"],
      "cwd": "{여기에_본인_홈경로}/obsidian-mcp-server"
    }
  }
}
```

예를 들어 홈 경로가 `/Users/me`라면:

```
{
  "mcpServers": {
```

```
    "obsidian-search": {
      "command": "/Users/me/obsidian-mcp-server/venv/bin/python",
      "args": ["/Users/me/obsidian-mcp-server/server.py"],
      "cwd": "/Users/me/obsidian-mcp-server"
    }
  }
}
```

> **중요**: `~` 는 여기서 안 된다. 반드시 절대 경로를 써야 한다.

4-3. 사용해보기

Claude Code를 실행하고 이렇게 물어봐라:

```
나: "내 핵심 가치 알려줘"
```

Claude가 자동으로 `search_notes("내 핵심 가치")` 를 호출하고, 관련 노트를 기반으로 답변한다.

확인하기

1. Claude Code에서 `/mcp` 명령으로 서버 상태 확인:
2. `obsidian-search` 가 등록되어 있어야 한다
3. 도구 2개가 보여야 한다: `search_notes` , `get_note`
4. 테스트 질문을 하고, Claude가 검색 도구를 사용하는지 확인

문제가 생기면

증상	원인	해결
MCP 서버가 안 나옴	`.mcp.json` 경로 오류	절대 경로 확인, `~` 대신 전체 경로 사용
서버 시작 에러	Python 패키지 미설치	`./venv/bin/pip install mcp[cli]`
검색은 되는데 결과가 이상함	indexer를 안 돌렸거나 오래됨	`./venv/bin/python indexer.py` 실행

증상	원인	해결
Claude가 도구를 안 씀	Claude Code를 재시작해야 함	Claude Code 종료 후 재시작

Part 5. Nightly Synapse -- 매일 밤 AI가 자동으로 노트를 분류하고 검증한다

왜 이게 필요한가?

하루에 메모를 여러 개 쓰면 분류가 밀린다. "이 메모는 핵심 사고인가, 휘발성 메모인가?"를 매번 판단하기 귀찮다. AI가 매일 밤 자동으로 "이건 핵심 노트로 승격시키는 게 좋겠습니다"라고 제안하고, 다른 AI가 그 판단을 교차검증한다. 최종 결정은 항상 당신이 한다.

5-1. 프로젝트 세팅

```
 # 볼트 안에 스크립트 폴더 생성
mkdir -p ~/Documents/Obsidian\ Vault/scripts/nightly-synapse
cd ~/Documents/Obsidian\ Vault/scripts/nightly-synapse

# package.json 생성
npm init -y

# 의존성 설치 (cron 스케줄러)
npm install node-cron
```

`package.json`을 아래와 같이 수정하라:

```
 {
  "name": "nightly-synapse",
  "version": "1.0.0",
  "description": "매일 저녁 볼트의 수정된 노트를 분류하고 검증하는 자동 시냅스",
  "type": "module",
  "scripts": {
    "start": "node scheduler.js",
    "run-now": "node run-now.js",
    "audit": "node run-audit.js"
  },
  "dependencies": {
    "node-cron": "^3.0.3"
  }
}
```

> **왜 `"type": "module"` 인가?** `import` / `export` 문법(ESM)을 쓰기 위해서다. `require` 를 쓰는 CommonJS보다 현대적인 방식.

5-2. detector.js -- 오늘 수정된 노트 감지

`detector.js` 파일을 만들어라:

```
// detector.js -- 오늘 수정된 .md 파일 감지
import fs from 'fs';
import path from 'path';

const IGNORE_DIRS = new Set([
  '.obsidian', '.smart-env', '.trash', '.git', '.claude',
  'scripts', 'Attachments', '4. Archives'
]);

const IGNORE_FILES = new Set([
  'CLAUDE.md', 'Claude_Project_Instruction.md'
]);

/**
 * 볼트 전체를 재귀 탐색하여 오늘 수정된 .md 파일만 반환
 */
export function getTodayModifiedNotes(vaultDir) {
  const today = new Date();
  today.setHours(0, 0, 0, 0);

  const results = [];
  walkDir(vaultDir, vaultDir, today, results);
  return results;
}

function walkDir(baseDir, currentDir, since, results) {
  let entries;
  try {
    entries = fs.readdirSync(currentDir, { withFileTypes: true });
  } catch { return; }

  for (const entry of entries) {
    if (entry.name.startsWith('.')) continue;

    const fullPath = path.join(currentDir, entry.name);
    const relativePath = path.relative(baseDir, fullPath);

    if (entry.isDirectory()) {
      const topDir = relativePath.split(path.sep)[0];
      if (IGNORE_DIRS.has(topDir) || IGNORE_DIRS.has(entry.name)) continue;
      walkDir(baseDir, fullPath, since, results);
    } else if (entry.name.endsWith('.md') && !IGNORE_FILES.has(entry.name)) {
```

```
      const stat = fs.statSync(fullPath);
      if (stat.mtime >= since) {
        results.push({
          file: entry.name,
          fullPath,
          relativePath,
          content: fs.readFileSync(fullPath, 'utf-8'),
        });
      }
    }
  }
}

/**
 * 기존 Slip-Box 노트 목록 -- classifier에 연결 참조용으로 전달
 */
export function getExistingSlipBoxNotes(vaultDir) {
  const slipBoxDir = path.join(vaultDir, '0. Slip-Box');
  if (!fs.existsSync(slipBoxDir)) return [];

  const notes = [];
  function walk(dir) {
    for (const entry of fs.readdirSync(dir, { withFileTypes: true })) {
      const full = path.join(dir, entry.name);
      if (entry.isDirectory()) walk(full);
      else if (entry.name.endsWith('.md')) notes.push(entry.name.replace('.md', ''));
    }
  }
  walk(slipBoxDir);
  return notes;
}
```

> **왜 `IGNORE_DIRS`가 있는가?** `.obsidian` 설정 파일이나 `scripts` 폴더, `Archives`를 분류할 필요는 없다. 노이즈를 줄여서 AI 토큰을 아끼는 거다.

5-3. classifier.js -- Claude CLI로 분류 판정

이게 핵심이다. Ground Truth 문서를 실행 시점에 읽어서 프롬프트에 넣는다. Ground Truth를 수정하면 코드 수정 없이 자동 반영된다.

`classifier.js` 파일을 만들어라:

```
// classifier.js -- Claude CLI로 분류 판정 (별도 API 키 불필요)
import fs from 'fs';
import path from 'path';

/**
```

```
 * Ground Truth 문서를 읽어서 분류 기준으로 사용
 */
function loadGroundTruth(vaultDir) {
  // 당신의 분류 기준 문서 경로를 여기에 넣으세요
  const gtPath = path.join(
    vaultDir,
    '{분류_기준_문서_경로}'  // 예: '2. Area/AI 설정/분류기준.md'
  );
  if (fs.existsSync(gtPath)) {
    return fs.readFileSync(gtPath, 'utf-8');
  }
  return '';
}

/**
 * 단일 노트를 분류 -- claude CLI의 -p (print) 모드 사용
 */
export async function classifyNote(note, existingSlipBoxNotes, vaultDir) {
  const groundTruth = loadGroundTruth(vaultDir);

  const prompt = `당신은 옵시디언 노트 분류 전문가입니다.
아래의 분류 기준 문서를 기준으로 노트를 분석하세요.

## 분류 기준
${groundTruth}

## 분석할 노트
- 파일명: ${note.file}
- 위치: ${note.relativePath}
- 내용:
${note.content.slice(0, 8000)}

## 기존 핵심 노트 목록 (연결 참고용)
${existingSlipBoxNotes.join('\n')}

## 작업 지침
1. 이 노트의 타입을 판정하세요 (핵심 사고 / 프로젝트 / 참고자료 / 휘발성).
2. 핵심 노트 승격 후보라면:
   - 연결할 기존 노트 (최소 2개)
   - 브릿지 키워드 (3~5개)
   를 제안하세요.
3. 승격이 아니라면 현재 위치가 적절한지 판단하세요.

## 응답 형식 (JSON만 출력, 마크다운 코드블록 없이)
{
  "type": "permanent | fleeting | project | reference | area",
  "reason": "분류 이유 한 줄",
  "action": "slip_box_promote | keep | move | merge",
  "current_location_ok": true,
  "suggested_move_to": "이동 제안 경로 (move일 때만)",
  "slip_box": {
    "connections": ["기존 노트명1", "기존 노트명2"],
    "bridge_keywords": ["키워드1", "키워드2", "키워드3"],
    "rewritten_content": "핵심 내용 요약 (200자 이내)"
  },
```

```
  "summary": "핵심 내용 2줄 요약"
}

slip_box 필드는 type이 "permanent"이고 action이 "slip_box_promote"일 때만 포함하세요.
`;

  // stdin으로 프롬프트 전달 (shell injection 방지)
  const { spawnSync } = await import('child_process');
  const result = spawnSync('claude', ['-p', '--output-format', 'text'], {
    input: prompt,
    encoding: 'utf-8',
    timeout: 120_000,
    maxBuffer: 10 * 1024 * 1024,
  });

  if (result.error || result.status !== 0) {
    console.error(`분류 실패: ${note.file}`, result.stderr);
    return null;
  }

  try {
    const output = result.stdout.trim();
    // JSON 파싱 (코드블록이 있으면 제거)
    const jsonStr = output.replace(/^```json?\n?/, '').replace(/\n?```$/, '');
    return JSON.parse(jsonStr);
  } catch (e) {
    console.error(`JSON 파싱 실패: ${note.file}`, e.message);
    return null;
  }
}
```

> **왜 `claude -p`를 쓰는가?** Claude CLI의 `-p` (print) 모드는 대화형 세션 없이 한 번에 질문하고 답변을 받는 모드다. 별도 API 키가 필요 없다 -- Claude Code 구독만 있으면 된다. 이게 이 시스템이 저렴한 이유다.

5-4. synapse.js -- 오케스트레이터

`synapse.js` 파일을 만들어라:

```
 // synapse.js -- 핵심 로직: 감지 -> 분류 -> 리포트
import { getTodayModifiedNotes, getExistingSlipBoxNotes } from './detector.js';
import { classifyNote } from './classifier.js';
import { generateAndSaveReport } from './report.js';
import { notifySynapseResult } from './notify.js';

const VAULT_DIR = process.env.VAULT_DIR
  || '{여기에_본인_볼트_절대경로}';
```

```
export async function runNightlySynapse() {
  console.log('오늘 수정된 노트 탐색 중...');

  // 1. 오늘 수정된 노트 수집
  const todayNotes = getTodayModifiedNotes(VAULT_DIR);
  if (todayNotes.length === 0) {
    console.log('오늘 수정된 노트 없음. 종료.');
    return;
  }
  console.log(`${todayNotes.length}개 노트 발견:`);
  todayNotes.forEach(n => console.log(`  - ${n.relativePath}`));

  // 2. 기존 Slip-Box 노트 목록
  const existingNotes = getExistingSlipBoxNotes(VAULT_DIR);
  console.log(`기존 핵심 노트: ${existingNotes.length}개`);

  // 3. 각 노트 분류 (순차 처리 -- rate limit 고려)
  const MAX_TOTAL_MS = 10 * 60 * 1000;
  const MAX_CONSECUTIVE_FAILURES = 3;
  const startTime = Date.now();
  let consecutiveFailures = 0;

  const results = [];
  for (const note of todayNotes) {
    if (Date.now() - startTime > MAX_TOTAL_MS) {
      console.log('전체 시간 제한(10분) 초과 -- 중단');
      break;
    }
    console.log(`분류 중: ${note.relativePath}`);
    try {
      const classification = await classifyNote(note, existingNotes, VAULT_DIR);
      if (classification) {
        results.push({ note, classification });
        consecutiveFailures = 0;
      } else {
        consecutiveFailures++;
      }
    } catch (err) {
      console.error(`에러: ${note.file}`, err.message);
      consecutiveFailures++;
    }
    if (consecutiveFailures >= MAX_CONSECUTIVE_FAILURES) {
      console.log(`연속 ${MAX_CONSECUTIVE_FAILURES}회 실패 -- 중단`);
      break;
    }
  }

  // 4. 리포트 생성 -> Inbox/
  if (results.length > 0) {
    const reportPath = generateAndSaveReport(results, VAULT_DIR);
    console.log(`리포트 생성: ${reportPath}`);

    // 5. macOS 알림
    notifySynapseResult(results.length, reportPath);
```

```
  }
}
```

5-5. report.js, notify.js, run-now.js

이 파일들은 리포트 생성과 알림을 담당한다. 여기서는 핵심 구조만 보여주겠다.

report.js -- Inbox/에 마크다운 리포트를 생성한다:

```
// report.js -- 분류 결과를 마크다운 리포트로 생성
import fs from 'fs';
import path from 'path';

export function generateAndSaveReport(results, vaultDir) {
  const today = new Date().toISOString().split('T')[0];
  const filename = `Nightly-Synapse_${today}.md`;
  const reportPath = path.join(vaultDir, 'Inbox', filename);

  let md = `# Nightly Synapse Report - ${today}\n\n`;
  md += `분석 노트: ${results.length}개\n\n---\n\n`;

  for (const { note, classification } of results) {
    md += `## ${note.file}\n`;
    md += `- 경로: ${note.relativePath}\n`;
    md += `- 분류: ${classification.type}\n`;
    md += `- 판정: ${classification.action}\n`;
    md += `- 이유: ${classification.reason}\n`;
    if (classification.slip_box) {
      md += `- 연결: ${classification.slip_box.connections?.join(', ')}\n`;
      md += `- 키워드: ${classification.slip_box.bridge_keywords?.join(', ')}\n`;
    }
    md += `\n---\n\n`;
  }

  // Inbox 폴더가 없으면 생성
  const inboxDir = path.join(vaultDir, 'Inbox');
  if (!fs.existsSync(inboxDir)) fs.mkdirSync(inboxDir, { recursive: true });

  fs.writeFileSync(reportPath, md, 'utf-8');
  return reportPath;
}
```

notify.js -- macOS 알림:

```
// notify.js -- macOS 알림 전송
import { execSync } from 'child_process';

export function notifySynapseResult(count, reportPath) {
  // 주의: LLM 출력을 직접 넣지 않는다 (command injection 방지)
  const safeCount = parseInt(count, 10) || 0;
```

```
  const message = `${safeCount}개 노트 분류 완료. Inbox에서 확인하세요.`;
  const title = 'Nightly Synapse';

  try {
    execSync(`osascript -e 'display notification "${message}" with title "${title}" s
  } catch {
    console.log(`알림: ${message}`);
  }
}
```

run-now.js -- 수동 실행:

```
// run-now.js -- 수동으로 Nightly Synapse 실행
import { runNightlySynapse } from './synapse.js';

console.log('=== Nightly Synapse 수동 실행 ===');
runNightlySynapse()
  .then(() => console.log('완료'))
  .catch(err => console.error('실패:', err));
```

5-6. 실행하기

```
# 수동 실행 (테스트)
cd ~/Documents/Obsidian\ Vault/scripts/nightly-synapse
node run-now.js
```

결과는 `Inbox/Nightly-Synapse_YYYY-MM-DD.md`에 생성된다.

5-7. 사용 흐름

```
매일 아침:
1. Inbox/Nightly-Synapse_*.md 확인
2. 핵심 노트 승격 제안이 맞으면 승인, 아니면 무시
3. 최종 결정은 항상 당신이 한다
```

확인하기

- [ ] `node run-now.js`를 실행하면 오늘 수정된 노트가 감지된다
- [ ] 각 노트에 대해 분류 결과 JSON이 출력된다
- [ ] `Inbox/` 폴더에 리포트 마크다운이 생성된다
- [ ] macOS 알림이 뜬다

문제가 생기면

증상	원인	해결
`command not found: claude`	Claude Code CLI 미설치	Claude Code 설치 필요
노트 0개 감지	오늘 수정한 노트가 없음	아무 노트나 수정 후 재실행
JSON 파싱 실패	Claude가 코드블록으로 감쌈	classifier.js의 코드블록 제거 로직 확인
시간 초과	노트가 너무 많거나 Claude 응답 느림	노트 수 제한 또는 timeout 증가

Part 6. CLAUDE.md -- AI가 내 볼트를 망치지 않게 하는 규칙

왜 이게 필요한가?

AI는 시키면 잘 한다. 문제는 "시키지 않은 것도 한다"는 거다. 노트를 수정하라고 하면 관련 노트까지 건드리고, 링크를 만들라고 하면 존재하지 않는 노트를 참조한다. 수년간 쌓은 노트를 AI가 망치지 않으려면 명시적인 규칙이 필요하다.

6-1. CLAUDE.md 구조 (템플릿)

볼트 루트에 `CLAUDE.md` 파일을 만들어라. Claude Code는 이 파일을 세션 시작 시 자동으로 읽는다.

```
 # CLAUDE.md -- [당신의 이름]의 옵시디언 볼트 운영 규칙

> 이 파일은 Claude가 세션 시작 시 읽는 절대 규칙이다.

---

## 1. 볼트 구조 요약

당신의 폴더 구조와 분류 기준을 여기에 적는다.
AI가 "이 노트를 어디에 넣어야 하는지" 판단할 때 참조하는 기준이다.

| 폴더 | 기준 | 비고 |
|------|------|------|
| 핵심 노트 | 원자성 있는 내 사고 | 가장 중요 |
| 프로젝트 | 구체적 산출물 + 마감 | 진행 중인 것들 |
| 영역 | 지속적 관리, 마감 없음 | 건강, 재정, 관계 등 |
| 참고 자료 | 외부 정보 | 나중에 쓸 것 |
| 보관함 | 완료/접은 것 | 나중에 다시 볼 수도 |

## 2. Active Context (매 세션 시작 시 업데이트)

현재 집중하고 있는 프로젝트를 여기에 적는다.
AI가 "지금 뭘 하고 있는지" 알아야 맥락에 맞는 답을 한다.

### 현재 집중 프로젝트
- 프로젝트 A: 현재 상태 설명
```

```
- 프로젝트 B: 현재 상태 설명

### 에너지 배분
- 프로젝트 A: 70%
- 프로젝트 B: 30%

## 3. 쓰기 제약 -- 볼트 오염 방지 규칙

### 3-1. 읽기 전용 기본값
- Claude는 기본적으로 볼트를 읽기만 한다.
- 쓰기/수정/삭제는 명시적 요청이 있을 때만 수행한다.
- 요청 없이 노트를 생성하거나 수정하지 않는다.

### 3-2. 제안 -> 컨펌 -> 실행 (3단계 원칙)
1. **제안**: 변경 사항을 먼저 텍스트로 보여준다
2. **컨펌**: 사용자가 승인한다
3. **실행**: 승인된 내용만 반영한다

### 3-3. 금지 행위
- **환각 연결 금지**: 존재하지 않는 [[링크]]를 만들지 않는다
- **자의적 확장 금지**: 지정된 노트만 건드린다 (Task Drift 방지)
- **구조 변경 금지**: 폴더/태그/프론트매터 규칙을 임의로 바꾸지 않는다

### 3-4. 안전장치
- 대량 작업(5개 이상 노트 수정) 시 단계별 컨펌으로 진행한다.

## 4. 세션 운영 규칙

### 4-1. 긴 대화에서 규칙 희석 방지
- 10턴 이상 길어지면 핵심 규칙이 희석된다.
- 대응: 작업 단위로 세션 분리

### 4-2. 컨텍스트 윈도우 관리
- 볼트 전체를 한번에 읽지 않는다.
- 필요한 폴더/파일만 읽는다.

## 5. 판단 기준

당신의 "업의 본질" 또는 "핵심 가치"를 한 줄로 적어라.
AI가 판단에 헷갈릴 때 이걸 기준으로 결정한다.

> "여기에 당신의 핵심 가치 한 문장"
```

> **왜 이렇게 하는가?** CLAUDE.md가 없으면 Claude는 "일반적인 AI"로 동작한다. 당신의 맥락을 모르고, 규칙도 모른다. 이 파일 하나가 "범용 AI"를 "나를 아는 AI"로 바꾸는 핵심이다.

6-2. Ground Truth (선택사항)

분류 기준이 복잡하다면, 별도 파일로 "분류 조건표"를 만들어라. Nightly Synapse의 classifier.js가 이 파일을 읽어서 분류한다.

포함할 내용: - 폴더 분류 If-Then 조건표 - 브릿지 키워드 목록 + 신호 규칙 - Edge Case 판정 사례

Ground Truth를 업데이트하면 classifier.js가 다음 실행 시 자동으로 반영한다. 코드 수정이 필요 없다.

6-3. 메모리 시스템 (선택사항)

Claude Code의 메모리 기능을 활용하면 세션이 바뀌어도 "나를 아는 AI"가 유지된다.

파일 위치: `~/.claude/projects/{볼트경로}/memory/`

- `MEMORY.md`: 세션 간 정보 유지 인덱스
- 개별 메모리 파일: 피드백, 참조, 프로젝트 맥락 등

Claude Code에게 "이걸 기억해"라고 하면 자동으로 메모리에 저장된다.

확인하기

- [ ] 볼트 루트에 `CLAUDE.md` 가 있다
- [ ] Claude Code를 시작하면 CLAUDE.md 내용을 인식한다
- [ ] "이 노트 수정해"라고 하면, Claude가 먼저 변경 내용을 보여주고 확인을 요청한다

Part 7. 텔레그램 파이프라인 -- 휴대폰으로 말하면 자동 저장

왜 이게 필요한가?

걸어가면서, 일하면서, 샤워하면서 떠오른 생각을 잡아야 한다. 옵시디언 앱을 켜서 폴더 찾고 제목 정하고 태그 달고... 이러면 생각이 날아간다. 텔레그램에 "결국 확률이다"라고 한 줄 치면, AI가 자동으로 분류하고 살을 붙여서 볼트에 저장한다.

7-1. 전체 흐름

```
 휴대폰 (텔레그램)
  -> 메시지 전송
Telegram Bot API (webhook)
  ->
API Gateway (HTTP API)
  ->
AWS Lambda (lambda_function.py)
  +-- 1. 메시지 수신 + 권한 확인 (본인만 사용 가능)
  +-- 2. 짧은 메모 반려 (너무 짧으면 돌려보냄)
  +-- 3. Bedrock Claude 호출
  |    +-- 분류 + 살 붙이기 + 브릿지 키워드 매핑
  +-- 4. DynamoDB에 임시 저장 (컨펌 대기)
  +-- 5. 텔레그램으로 미리보기 + 버튼 전송
  |    +-- [저장] [Inbox로] [취소]
  +-- 6. 버튼 클릭시 -> GitHub API로 커밋
       ->
GitHub Repository (private)
  -> sync_from_github.sh (30분마다)
Obsidian Vault에 노트 등장
```

7-2. Step 1: 텔레그램 봇 생성 (5분)

1. 텔레그램에서 `@BotFather` 검색
2. `/newbot` 입력

3. 봇 이름 입력 (예: `나의 옵시디언 봇`)
4. 봇 username 입력 (예: `my_obsidian_bot`)
5. **봇 토큰** 복사해서 안전한 곳에 메모 -> `{여기에_본인_텔레그램_봇_토큰}`

7-3. Step 2: GitHub 저장소 + 토큰 (10분)

1. GitHub에서 새 **Private** 저장소 생성 (예: `obsidian-vault`)
2. 로컬 볼트를 이 저장소에 push:

```
 cd ~/Documents/Obsidian\ Vault
git init
git remote add origin https://github.com/{여기에_본인_GitHub_유저명}/obsidian-vault.git
git add -A
git commit -m "initial commit"
git push -u origin main
```

1. GitHub -> Settings -> Developer settings -> Personal access tokens -> Fine-grained tokens
2. 새 토큰 생성:
3. Repository access: 위에서 만든 저장소만 선택
4. Permissions: Contents -> Read and Write
5. 토큰 복사 -> `{여기에_본인_GitHub_토큰}`

7-4. Step 3: AWS 설정 (20분)

3-1. Bedrock 모델 액세스 활성화

1. AWS 콘솔 -> Amazon Bedrock -> Model access
2. Claude 모델 활성화 요청 (Claude Sonnet 또는 Opus)
3. 승인까지 몇 분 소요

3-2. DynamoDB 테이블 생성

1. AWS 콘솔 -> DynamoDB -> 테이블 생성
2. 테이블 이름: `obsidian-pending-notes`
3. 파티션 키: `id` (문자열)

4. 나머지 기본값

3-3. Lambda 함수 생성

1. AWS 콘솔 -> Lambda -> 함수 생성
2. 함수 이름: `obsidian-telegram-pipeline`
3. 런타임: **Python 3.12**
4. 아키텍처: **arm64** (20% 저렴)
5. 일반 구성 수정:
6. 타임아웃: **90초**
7. 메모리: **256MB**

3-4. Lambda 코드

`lambda_function.py` 의 핵심 구조:

```
 """
텔레그램 -> Bedrock Claude -> GitHub -> Obsidian 파이프라인
AWS Lambda 핸들러
"""

import json
import os
import re
import base64
import urllib.request
import urllib.parse
from datetime import datetime, timezone, timedelta

import boto3

# === 환경변수 (Lambda 콘솔에서 설정) ===
TELEGRAM_BOT_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
GITHUB_TOKEN = os.environ["GITHUB_TOKEN"]
GITHUB_REPO = os.environ.get("GITHUB_REPO", "{여기에_본인_GitHub유저/저장소}")
GITHUB_BRANCH = os.environ.get("GITHUB_BRANCH", "main")
BEDROCK_MODEL_ID = os.environ.get("BEDROCK_MODEL_ID", "us.anthropic.claude-sonnet-4-2(
AWS_BEDROCK_REGION = os.environ.get("AWS_BEDROCK_REGION", "us-east-1")
ALLOWED_CHAT_ID = int(os.environ.get("ALLOWED_CHAT_ID", "{여기에_본인_텔레그램_Chat_ID}"))
DYNAMO_TABLE = os.environ.get("DYNAMO_TABLE", "obsidian-pending-notes")

KST = timezone(timedelta(hours=9))

# === 브릿지 키워드 사전 ===
# bridge_keywords.py와 동일한 내용을 여기에도 넣어야 한다.
# 한쪽을 수정하면 반드시 다른 쪽도 동기화하세요.
```

```
BRIDGE_KEYWORDS = {
    # ... bridge_keywords.py와 동일한 딕셔너리 ...
}

# === 시스템 프롬프트 (템플릿) ===
SYSTEM_PROMPT = """당신은 사용자의 제2의 뇌(Second Brain)입니다.
사용자가 텔레그램으로 보낸 짧은 메모를 분석하여:

1. 의도를 파악하고
2. 적절한 폴더로 분류하고
3. 살을 붙여서 깊이 있는 노트로 만듭니다.

## 분류 기준
- 핵심 사고: 원자성 있는 자기 사고 -> Slip-Box 승격 후보
- 프로젝트: 산출물+마감 있으면 해당 프로젝트 폴더
- 학습/참고: 외부 지식 정리 -> Resources
- 일상 메모: 구체적 맥락 부족 -> Inbox

## 브릿지 키워드 매핑
BRIDGE_KEYWORDS 사전의 신호 단어가 감지되면 해당 키워드를 노트에 태깅합니다.
3~5개 사이로 태깅하세요.

## 출력 형식 (JSON)
{
  "title": "노트 제목",
  "folder": "저장할 폴더 경로",
  "content": "마크다운 형식의 노트 본문",
  "tags": ["PKM/카테고리"],
  "bridge_keywords": ["키워드1", "키워드2", "키워드3"],
  "confidence": 0.85,
  "summary": "한 줄 요약"
}

confidence가 0.7 미만이면 Inbox에 저장합니다.
"""

# === AWS 클라이언트 ===
bedrock_client = boto3.client("bedrock-runtime", region_name=AWS_BEDROCK_REGION)
dynamodb = boto3.resource("dynamodb", region_name=AWS_BEDROCK_REGION)
pending_table = dynamodb.Table(DYNAMO_TABLE)


def lambda_handler(event, context):
    """Lambda 진입점"""
    body = json.loads(event.get("body", "{}"))

    if "callback_query" in body:
        return handle_callback(body["callback_query"])
    elif "message" in body:
        return handle_new_message(body["message"])

    return {"statusCode": 200}


def handle_new_message(message):
```

```
    """새 메시지 처리"""
    chat_id = message["chat"]["id"]

    # 권한 확인: 본인만 사용 가능
    if chat_id != ALLOWED_CHAT_ID:
        return {"statusCode": 200}

    text = message.get("text", "").strip()
    if not text:
        return {"statusCode": 200}

    # /start 명령어
    if text == "/start":
        send_telegram(chat_id, "연결 완료. 메모를 보내면 자동으로 분류/저장합니다.")
        return {"statusCode": 200}

    # 너무 짧은 메모 반려
    if is_too_short(text):
        send_telegram(chat_id, "메모가 너무 짧습니다. 맥락을 좀 더 붙여서 다시 보내주세요.")
        return {"statusCode": 200}

    # Bedrock Claude 호출
    result = call_bedrock_claude(text)
    if not result:
        send_telegram(chat_id, "처리 중 오류가 발생했습니다.")
        return {"statusCode": 200}

    # DynamoDB에 임시 저장 (컨펌 대기)
    pending_id = save_pending(result)

    # 텔레그램으로 미리보기 + 버튼 전송
    preview = format_preview(result)
    send_telegram_with_buttons(chat_id, preview, pending_id)

    return {"statusCode": 200}


def handle_callback(callback):
    """버튼 클릭 처리 (저장/Inbox로/취소)"""
    # ... 버튼에 따라 GitHub 커밋 또는 취소 처리 ...
    pass


def is_too_short(text):
    """3줄 이하 + 맥락 없음 -> 반려"""
    lines = [l.strip() for l in text.split('\n') if l.strip()]
    return len(lines) <= 1 and len(text) < 20


def call_bedrock_claude(user_text):
    """Bedrock Claude 호출 -- Context Caching 적용"""
    response = bedrock_client.invoke_model(
        modelId=BEDROCK_MODEL_ID,
        body=json.dumps({
            "anthropic_version": "bedrock-2023-05-31",
```

```
            "max_tokens": 4096,
            "system": [
                {
                    "type": "text",
                    "text": SYSTEM_PROMPT,
                    "cache_control": {"type": "ephemeral"}  # 캐싱으로 90% 비용 절감
                }
            ],
            "messages": [
                {"role": "user", "content": user_text}
            ]
        }),
        contentType="application/json"
    )
    # 응답 파싱
    result = json.loads(response["body"].read())
    return json.loads(result["content"][0]["text"])


def commit_to_github(filepath, content, commit_message):
    """GitHub API로 파일 커밋"""
    url = f"https://api.github.com/repos/{GITHUB_REPO}/contents/{filepath}"
    data = {
        "message": commit_message,
        "content": base64.b64encode(content.encode()).decode(),
        "branch": GITHUB_BRANCH
    }
    req = urllib.request.Request(
        url, method="PUT",
        data=json.dumps(data).encode(),
        headers={
            "Authorization": f"token {GITHUB_TOKEN}",
            "Content-Type": "application/json"
        }
    )
    urllib.request.urlopen(req)


def send_telegram(chat_id, text):
    """텔레그램 메시지 전송"""
    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    data = urllib.parse.urlencode({
        "chat_id": chat_id,
        "text": text,
        "parse_mode": "Markdown"
    }).encode()
    urllib.request.urlopen(url, data)
```

> **이건 핵심 구조를 보여주는 템플릿이다.** 실제로는 DynamoDB 저장/조회, 콜백 버튼 처리, 에러 핸들링 등이 더 들어간다. 이 구조를 바탕으로 Claude Code에게 "이 구조대로 완전한 코드를 만들어

> 줘"라고 하면 된다.

3-5. 환경변수 설정

Lambda -> 구성 -> 환경변수:

키	값
`TELEGRAM_BOT_TOKEN`	`{여기에_본인_텔레그램_봇_토큰}`
`GITHUB_TOKEN`	`{여기에_본인_GitHub_토큰}`
`GITHUB_REPO`	`{GitHub유저명}/{저장소명}`
`GITHUB_BRANCH`	`main`
`BEDROCK_MODEL_ID`	`us.anthropic.claude-sonnet-4-20250514`
`AWS_BEDROCK_REGION`	`us-east-1`
`ALLOWED_CHAT_ID`	`{여기에_본인_텔레그램_Chat_ID}`
`DYNAMO_TABLE`	`obsidian-pending-notes`

> **Chat ID 찾는 법**: 텔레그램에서 `@userinfobot` 에게 아무 메시지를 보내면 알려준다.

3-6. IAM 역할에 권한 추가

Lambda -> 구성 -> 권한 -> 실행 역할 클릭 -> 인라인 정책 추가:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "bedrock:InvokeModel",
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "dynamodb:PutItem",
```

```
                "dynamodb:GetItem",
                "dynamodb:DeleteItem"
            ],
            "Resource": "arn:aws:dynamodb:*:*:table/obsidian-pending-notes"
        }
    ]
}
```

3-7. API Gateway 트리거 추가

1. Lambda -> 트리거 추가 -> API Gateway
2. API 유형: **HTTP API** (REST보다 저렴)
3. 보안: **열기** (텔레그램 webhook용)
4. 생성된 **API 엔드포인트 URL** 복사

7-5. Step 4: Webhook 연결 (1분)

```
curl "https://api.telegram.org/bot{여기에_본인_텔레그램_봇_토큰}/setWebhook?url={여기에_API_G
```

응답에 `"ok": true`가 나오면 성공.

7-6. Step 5: 배포 스크립트

`~/telegram-obsidian-pipeline/deploy.sh` 파일을 만들어라:

```
#!/bin/bash
# Lambda 배포 스크립트
# boto3는 Lambda 런타임에 기본 포함이므로 별도 패키징 불필요

set -e

FUNCTION_NAME="obsidian-telegram-pipeline"
REGION="us-east-1"
ZIP_FILE="lambda_package.zip"

echo "=== Lambda 배포 패키지 생성 ==="

rm -f "$ZIP_FILE"
zip "$ZIP_FILE" lambda_function.py

echo "=== 패키지 생성 완료: $ZIP_FILE ===="

if aws lambda get-function --function-name "$FUNCTION_NAME" --region "$REGION" 2>/dev/
    echo "=== 기존 함수 업데이트 ===="
```

```
    aws lambda update-function-code \
        --function-name "$FUNCTION_NAME" \
        --zip-file "fileb://$ZIP_FILE" \
        --region "$REGION"
    echo "=== 업데이트 완료 ===" 
else
    echo "Lambda 함수가 아직 없습니다. AWS 콘솔에서 먼저 생성하세요."
fi
```

```
chmod +x ~/telegram-obsidian-pipeline/deploy.sh
```

7-7. 비용 구조

서비스	과금 기준	예상 비용 (월 100건)
Lambda	요청당 + 실행 시간	~$0 (프리티어)
API Gateway	요청당	~$0 (프리티어)
Bedrock Claude	입력 $15/M토큰, 출력 $75/M토큰	~$3.00
Context Caching 절감	시스템 프롬프트 캐싱 (90% 절감)	-$1.50
DynamoDB	읽기/쓰기 요청	~$0 (프리티어)
합계		**월 ~$2**

확인하기

1. 텔레그램 봇에게 메시지 보내기: `의사결정에서 중요한 건 결과가 아니라 과정이다. 좋은 과정은 장기적으로 좋은 결과를 만든다.`
2. 기대 결과:
3. 봇이 응답: 분류 결과 + 저장 경로 + 미리보기
4. [저장] 버튼 클릭
5. GitHub에 커밋 생성
6. 30분 내 Obsidian에 노트 등장

문제가 생기면

증상	원인	해결
봇이 응답 없음	Webhook 미연결	`curl "https://api.telegram.org/bot{토큰}/getWebhookInfo"` 로 확인
"Access Denied"	Bedrock 모델 미활성화	AWS Bedrock 콘솔에서 모델 액세스 확인
GitHub 커밋 실패	토큰 권한 부족	Contents: Read and Write 권한 확인
Obsidian에 안 나타남	Git pull 미설정	Part 8의 동기화 스크립트 설정
Lambda 타임아웃	Claude 응답이 느림	타임아웃을 90초 이상으로 설정

Part 8. 동기화 & 백업 -- 데이터를 안전하게 지키기

왜 이게 필요한가?

텔레그램 파이프라인이 GitHub에 커밋하면, 그걸 로컬 볼트로 가져와야 한다. 반대로 옵시디언에서 직접 쓴 메모도 GitHub에 올려야 한다. 그리고 모든 데이터에는 백업이 필요하다. 이 두 가지를 자동화한다.

8-1. GitHub 양방향 동기화 스크립트

`~/ObsidianBackup/sync_from_github.sh` 파일을 만들어라:

```
 #!/bin/bash
# GitHub <-> Obsidian Vault 양방향 동기화 스크립트
# 1. 로컬 변경 -> GitHub push
# 2. GitHub 변경 -> 로컬 pull (텔레그램 메모 등 수신)

set -e

VAULT="{여기에_본인_볼트_절대경로}"   # 예: "/Users/me/Documents/Obsidian Vault"
LOG="{여기에_로그파일_경로}"            # 예: "/Users/me/ObsidianBackup/sync.log"

cd "$VAULT" || exit 1

# 1. 로컬 변경사항을 GitHub에 push
if [ -n "$(git status --porcelain)" ]; then
    git add -A
    # 민감 파일이 stage에 올라왔으면 제거
    git diff --cached --name-only | grep -iE '\.env$|credentials|secret|\.key$|\.pem$
    git commit -m "auto-sync: $(date '+%Y-%m-%d %H:%M') 볼트 변경사항 동기화" --quiet 2>/de
    if ! git push origin main --quiet 2>/dev/null; then
        echo "$(date '+%Y-%m-%d %H:%M'): push 실패" >> "$LOG"
        exit 1
    fi
    echo "$(date '+%Y-%m-%d %H:%M'): push 완료" >> "$LOG"
fi

# 2. GitHub 변경사항을 로컬로 pull (텔레그램 메모 등)
```

```
if ! git pull --no-rebase origin main --quiet 2>/dev/null; then
    echo "$(date '+%Y-%m-%d %H:%M'): pull 실패 (충돌 가능성). 수동 확인 필요." >> "$LOG"
    # 충돌 시 안전 복원
    git merge --abort 2>/dev/null || true
    exit 1
fi

echo "$(date '+%Y-%m-%d %H:%M'): 동기화 완료" >> "$LOG"
```

```
# 실행 권한 부여
chmod +x ~/ObsidianBackup/sync_from_github.sh
```

> **왜 `.env`를 자동 제거하는가?** git add -A를 하면 모든 파일이 올라간다. 실수로 API 키가 든 .env 파일이 GitHub에 올라가면 큰일난다. 이 안전장치가 그걸 막는다.

8-2. 일일 백업 스크립트

`~/ObsidianBackup/backup_vault.sh` 파일을 만들어라:

```
#!/bin/bash
# Obsidian Vault 일일 백업 스크립트
# 압축 파일명: ObsidianVault_YYYY-MM-DD.tar.gz

VAULT="{여기에_본인_볼트_절대경로}"
BACKUP_DIR="{여기에_백업_폴더_경로}"   # 예: "/Users/me/ObsidianBackup"
DATE=$(date +%Y-%m-%d)
FILENAME="ObsidianVault_${DATE}.tar.gz"

# 압축 백업
tar -czf "${BACKUP_DIR}/${FILENAME}" -C "$(dirname "$VAULT")" "$(basename "$VAULT")"

# 7일 이상 된 백업 자동 삭제 (디스크 관리)
find "$BACKUP_DIR" -name "ObsidianVault_*.tar.gz" -mtime +7 -delete

echo "$(date): Backup completed - ${FILENAME}"
```

```
chmod +x ~/ObsidianBackup/backup_vault.sh
```

8-3. 자동 실행 (macOS LaunchAgent)

macOS에서는 LaunchAgent로 스크립트를 자동 실행할 수 있다. cron보다 안정적이다.

30분마다 동기화

`~/Library/LaunchAgents/com.user.obsidian-github-sync.plist`:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/Prope
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.user.obsidian-github-sync</string>
    <key>ProgramArguments</key>
    <array>
        <string>{여기에_홈경로}/ObsidianBackup/sync_from_github.sh</string>
    </array>
    <key>StartInterval</key>
    <integer>1800</integer>
    <key>RunAtLoad</key>
    <true/>
    <key>StandardOutPath</key>
    <string>{여기에_홈경로}/ObsidianBackup/sync-stdout.log</string>
    <key>StandardErrorPath</key>
    <string>{여기에_홈경로}/ObsidianBackup/sync-stderr.log</string>
</dict>
</plist>
```

매일 21:30 백업

`~/Library/LaunchAgents/com.user.obsidian-backup.plist`:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/Prope
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.user.obsidian-backup</string>
    <key>ProgramArguments</key>
    <array>
        <string>{여기에_홈경로}/ObsidianBackup/backup_vault.sh</string>
    </array>
    <key>StartCalendarInterval</key>
    <dict>
        <key>Hour</key>
        <integer>21</integer>
        <key>Minute</key>
        <integer>30</integer>
    </dict>
</dict>
</plist>
```

매일 21:30 Nightly Synapse

`~/Library/LaunchAgents/com.user.nightly-synapse.plist`:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/Prope
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.user.nightly-synapse</string>
    <key>ProgramArguments</key>
    <array>
        <string>/usr/local/bin/node</string>
        <string>{여기에_볼트경로}/scripts/nightly-synapse/run-now.js</string>
    </array>
    <key>StartCalendarInterval</key>
    <dict>
        <key>Hour</key>
        <integer>21</integer>
        <key>Minute</key>
        <integer>30</integer>
    </dict>
    <key>WorkingDirectory</key>
    <string>{여기에_볼트경로}/scripts/nightly-synapse</string>
</dict>
</plist>
```

LaunchAgent 등록

```
# 로드 (활성화)
launchctl load ~/Library/LaunchAgents/com.user.obsidian-github-sync.plist
launchctl load ~/Library/LaunchAgents/com.user.obsidian-backup.plist
launchctl load ~/Library/LaunchAgents/com.user.nightly-synapse.plist

# 상태 확인
launchctl list | grep com.user
```

> **Linux/Windows를 쓴다면**: LaunchAgent 대신 cron (Linux) 또는 Task Scheduler (Windows)를 사용하라.

확인하기

```
# 동기화 테스트
~/ObsidianBackup/sync_from_github.sh

# 백업 테스트
~/ObsidianBackup/backup_vault.sh
```

```
# 백업 파일 확인
ls -la ~/ObsidianBackup/ObsidianVault_*.tar.gz
```

문제가 생기면

증상	원인	해결
push 실패	GitHub 인증 문제	`git config credential.helper store` 후 재인증
pull 충돌	같은 파일을 양쪽에서 수정	`git merge --abort` 후 수동 해결
LaunchAgent 안 됨	plist 로드 안 됨	`launchctl load` 재실행
백업 용량 과다	.git 폴더 포함됨	tar에 `--exclude=.git` 추가

Part 9. 개발 하네스 -- AI가 코드를 짤 때 품질을 잡아주는 시스템 (선택)

> 이 파트는 시스템을 직접 수정하거나 확장할 때 필요하다. 처음 구축할 때는 건너뛰어도 된다.

왜 이게 필요한가?

Claude Code에게 코드를 시키면 잘 짠다. 근데 가끔 보안 허점을 남기거나, 테스트 없이 넘어가거나, `--no-verify` 로 git hook을 건너뛴다. 하네스는 이런 걸 자동으로 잡아주는 "품질 안전망"이다.

9-1. 하네스 구성 요소

구성 요소	위치	역할
규칙 (Rules)	`~/.claude/rules/`	코딩 스타일, 보안, 테스트 규칙
훅 (Hooks)	`~/.claude/settings.json`	특정 동작 시 자동 점검
에이전트	`~/.claude/agents/`	전문 역할 AI (설계자, 리뷰어 등)
스킬	`~/.claude/skills/`	반복 워크플로우 자동화

9-2. 훅 설정 (settings.json)

`~/.claude/settings.json` 에 추가:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
```

```
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'INPUT=$(cat); if echo \"$INPUT\" | grep -q \"\\-\\-no
          }
        ],
        "description": "Block --no-verify flag to protect git hooks"
      },
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c 'INPUT=$(cat); if echo \"$INPUT\" | grep -qE \"git com
          }
        ],
        "description": "Pre-commit quality check: detect secrets and debug statements"
      }
    ],
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"작업이 완료되었습니다.\" with
          }
        ]
      }
    ]
  }
}
```

훅이 하는 일: - `--no-verify` 차단: git hook을 건너뛰지 못하게 막는다 - 시크릿 감지: 커밋에 API 키나 디버그 구문이 포함되면 경고 - 작업 완료 알림: Claude가 작업을 끝내면 macOS 알림

9-3. 규칙 파일

`~/.claude/rules/security.md`:

```
# Security Guidelines

Before ANY commit:
- [ ] No hardcoded secrets (API keys, passwords, tokens)
- [ ] All user inputs validated
- [ ] Error messages don't leak sensitive data

## Secret Management
- NEVER hardcode secrets in source code
- ALWAYS use environment variables
```

`~/.claude/rules/coding-style.md`:

```
 # Coding Style

## Core Principles
- KISS -- Simplest solution that works
- DRY -- No copy-paste
- YAGNI -- Don't build before needed

## Immutability
ALWAYS create new objects, NEVER mutate existing ones.
```

9-4. 사용 방법

이건 설정해두면 자동으로 동작한다. 별도로 뭘 할 필요 없다.

Claude Code가 코드를 짤 때: 1. 커밋하려면 훅이 자동으로 시크릿 체크 2. `--no-verify` 쓰려 하면 자동 차단 3. 작업 끝나면 알림

더 자세한 내용은 Claude Code 공식 문서를 참고하라.

Part 10. 일상 운영 가이드 -- 매일 이것만 하면 된다

왜 이게 필요한가?

시스템을 만드는 건 한 번이다. 쓰는 건 매일이다. 여기서는 "매일 뭘 해야 하는지"와 "뭔가 잘못됐을 때 뭘 해야 하는지"를 정리한다.

10-1. 매일 아침 (5분)

순서	할 것
1	`Inbox/Nightly-Synapse_*.md` 확인 -> 승격 제안 승인/거부
2	`Inbox/Daily-Audit_*.md` 확인 -> AI 작업 품질 점검

이게 끝이다. 5분이면 된다.

10-2. 메모를 쓰거나 수정했을 때

노트를 직접 수정했으면 인덱서를 돌려라. 동기화 스크립트가 자동으로 GitHub에 올리지만, 검색 DB는 수동 업데이트가 필요하다.

```
cd ~/obsidian-mcp-server && ./venv/bin/python indexer.py
```

> 이것도 LaunchAgent로 자동화할 수 있지만, 처음에는 수동으로 돌리는 걸 추천한다. 시스템이 어떻게 작동하는지 감을 잡은 후에 자동화하라.

10-3. 상황별 대응 체크리스트

상황	할 것
검색 결과가 이상할 때	`bridge_keywords.py` 에 키워드 추가 -> `./venv/bin/python indexer.py`
볼트 규칙을 바꿨을 때	`CLAUDE.md` 업데이트 (Nightly Synapse는 Ground Truth 자동 반영)
Lambda 코드를 수정했을 때	`cd ~/telegram-obsidian-pipeline && ./deploy.sh`
동기화가 안 될 때	`~/ObsidianBackup/sync_from_github.sh` 수동 실행
스케줄러가 꺼져 있을 때	`launchctl load ~/Library/LaunchAgents/com.user.*.plist`
텔레그램 봇이 안 됨	`curl "https://api.telegram.org/bot{토큰}/getWebhookInfo"` 확인
브릿지 키워드를 추가했을 때	`bridge_keywords.py` + `lambda_function.py` 양쪽 동기화 -> 인덱서 + Lambda 배포

10-4. 주간 체크 (선택)

순서	할 것
1	Inbox 폴더 정리 -- 분류 안 된 노트 직접 분류
2	브릿지 키워드 점검 -- 새로 추가할 키워드 있는지
3	백업 확인 -- `ls ~/ObsidianBackup/ObsidianVault_*.tar.gz`

10-5. 전체 파일 위치 한눈에 보기

파일/폴더	경로	역할
`indexer.py`	`~/obsidian-mcp-server/`	증분 인덱싱 엔진
`search.py`	`~/obsidian-mcp-server/`	4중 가중치 검색 엔진

파일/폴더	경로	역할
`bridge_keywords.py`	`~/obsidian-mcp-server/`	브릿지 키워드 사전
`server.py`	`~/obsidian-mcp-server/`	MCP 서버 (Claude 연동)
`vault.lancedb/`	`~/obsidian-mcp-server/`	벡터 DB
`.mcp.json`	`~/.claude/`	MCP 서버 등록
`lambda_function.py`	`~/telegram-obsidian-pipeline/`	텔레그램 핵심 코드
`deploy.sh`	`~/telegram-obsidian-pipeline/`	Lambda 배포
`synapse.js`	`볼트/scripts/nightly-synapse/`	자동 분류 오케스트레이터
`classifier.js`	`볼트/scripts/nightly-synapse/`	Claude CLI 분류 판정
`detector.js`	`볼트/scripts/nightly-synapse/`	오늘 수정된 노트 감지
`CLAUDE.md`	볼트 루트	볼트 운영 규칙
`sync_from_github.sh`	`~/ObsidianBackup/`	양방향 동기화
`backup_vault.sh`	`~/ObsidianBackup/`	일일 백업
`*.plist` (3~4개)	`~/Library/LaunchAgents/`	자동 스케줄

마치며

이 시스템의 본질은 4줄로 요약된다:

1. **내 노트를 숫자로 바꾸고** (임베딩)
2. **숫자끼리 비교해서 비슷한 걸 찾고** (벡터 검색)
3. **내 기준으로 중요한 걸 위로 올리고** (4중 가중치)
4. **Claude가 이걸 도구로 쓸 수 있게 연결했다** (MCP)

그 위에 **자동 분류(Nightly Synapse)**, **모바일 입력(텔레그램)**, **교차검증(Daily Audit)**, **품질 관리(하네스)**가 얹혀있다.

기술적 디테일은 몰라도 된다. "도구가 어떻게 작동하는지 아는 것"보다 "도구를 언제 어떻게 쓰는지 아는 것"이 더 중요하다.

Part 1~8까지 따라하면 기본 시스템 완성. Part 9는 확장할 때.

구축 순서 추천: 1. Part 2~4 (임베딩 + 검색 + MCP) -- 핵심. 이것만 해도 "나를 아는 AI"가 된다. 2. Part 6 (CLAUDE.md) -- 규칙. AI가 삽질하는 걸 막는다. 3. Part 8 (동기화 + 백업) -- 안전망. 4. Part 7 (텔레그램) -- 편의성. 모바일 입력. 5. Part 5 (Nightly Synapse) -- 자동화. 매일 밤 분류. 6. Part 9 (하네스) -- 선택. 개발할 때.

처음부터 전부 만들려고 하지 마라. 2~4를 먼저 만들고 1주일 써봐라. 그 다음에 필요한 걸 하나씩 추가하라. "Do it once manually, twice consider, thrice automate."